microBee

RV32IMC Soft-Core Microcontroller IP

Technical Reference

Document number: microBee-TR-001

Revision: P1.2

Product type:	Synthesizable SystemVerilog IP
Architecture:	32-bit RISC-V RV32IMC
Reference profile:	microBee-REF32
Audience:	Integrators and firmware developers

Scope. microBee is not a packaged semiconductor. It is a microcontroller IP instantiated in an FPGA or ASIC flow. This technical reference defines normative processor behavior, ports, bus protocol, memory and register maps, interrupt model, and configuration rules that exist in the shipped RTL. Voltage, temperature, I/O standard, power, and maximum clock frequency are target-dependent and are taken from the selected device, board, constraints, and implementation report.

Document Usage Rules

This document is the normative product interface contract for microBee. Pedagogical narrative and laboratory procedures belong in separate course materials and are outside the scope of this reference.

Table 1: Data classes

Class	Meaning
Fixed	Property identical across all compatible microBee instances.
Configurable	Selected at synthesis or SoC integration; fixed for a given bitstream.
REF32 profile	Value frozen by the ready-to-use microBee-REF32 profile.
Target-dependent	Determined by FPGA, board, memory macro, tool version, or timing constraint.
Reserved	Reserved for future use; software shall write zero unless otherwise stated.

Normative source order. The architecture decision contract and the shipped SystemVerilog sources define what the product implements. For ISA and machine-mode behavior, RISC-V International ratified specifications apply [1, 2]. This datasheet defines microBee-specific ports, registers, and integration details. A profile manifest may freeze only values that this document leaves configurable. Features that are not present in the current RTL are not described as product capabilities.

Contents

Document Usage Rules **i**

1 Document Control **1**
1.1 Identification . . . 1
1.2 Normative language . . . 1
1.3 Related documents . . . 1

2 Product Overview **2**
2.1 Description . . . 2
2.2 Key properties . . . 2
2.3 Delivered product items . . . 2

3 Reference Profile microBee-REF32 **3**
3.1 Profile code . . . 3

4 Functional Description **4**
4.1 System organization . . . 4
4.2 Clock and reset . . . 4
4.3 Boot model . . . 4

5 Programmer's Model **5**
5.1 Integer registers . . . 5
5.2 Shadow GPR bank . . . 5
5.3 Endianness . . . 5

6 Pipeline **6**
6.1 Organization . . . 6
6.2 Branch prediction . . . 6
6.3 Flush scope . . . 6

7 Instruction Set **7**
7.1 Supported extensions . . . 7
7.2 Latency and throughput . . . 7
7.3 Fence . . . 7

8 Interfaces **8**
8.1 Top-level / SoC ports . . . 8
8.2 Interrupt wiring . . . 8

9 Native Request-Response Bus **9**

9.1 Channel model . . . 9
9.2 Posted store versus synchronous response . . . 9
9.3 Outstanding requests and ordering . . . 9
9.4 Handshake stability (implemented slave/core rules) . . . 9

10 Memory Map **11**
10.1 Address model . . . 11
10.2 microBee-REF32 map . . . 11
10.3 Decode priority . . . 11
10.4 Data-path access to the IMEM window . . . 12
10.5 Linker placement . . . 12
10.6 Startup image and `.data` initialization . . . 12

11 Local Memory **13**
11.1 IMEM . . . 13
11.2 DMEM . . . 13
11.3 Coherence . . . 13

12 CSR and Traps **14**
12.1 Implemented machine CSRs . . . 14
12.2 WARL and field behavior (implemented) . . . 14
12.3 Exception `mepc`/`mtval` summary . . . 15
12.4 Interrupt causes . . . 15
12.5 Trap entry and return . . . 15
12.6 Counter reads . . . 15

13 CLINT **16**
13.1 Overview . . . 16
13.2 MSIP and timer interrupt . . . 16
13.3 Reset and time base . . . 16
13.4 64-bit compare update order . . . 16

14 PLIC **17**
14.1 Overview . . . 17
14.2 REF32 source allocation . . . 17
14.3 SOURCE_MODE and INPUT_STATE packing . . . 18
14.4 Selection rules . . . 18
14.5 Trigger modes . . . 18
14.6 Input synchronization . . . 18

15 GPIO **19**
15.1 Overview . . . 19
15.2 Register map . . . 19
15.3 Input synchronization . . . 19

16 General-Purpose Timer **20**
16.1 Relation to CLINT . . . 20
16.2 Operation . . . 20

16.3 Registers . . . 20
16.4 CTRL and STATUS bits . . . 21

17 UART **22**
17.1 Native UART versus board example . . . 22
17.2 Native UART function . . . 22
17.3 STATUS and CTRL (native) . . . 23
17.4 IRQ_STATUS and FIFO_LEVEL . . . 23

18 Performance Monitor **24**
18.1 Overview . . . 24
18.2 Control . . . 24
18.3 Counter offsets (live) . . . 24

19 AXI4-Lite Master **25**
19.1 Scope . . . 25
19.2 Bridge contract (implemented) . . . 25

20 Configuration Parameters **26**
20.1 Dependencies . . . 26

21 Clocking, Timing, and Resources **28**
21.1 Logical timing . . . 28
21.2 REF32 operating point . . . 28
21.3 Resource and electrical limits . . . 28

22 Software Integration Contract **29**
22.1 Toolchain . . . 29
22.2 Startup order . . . 29
22.3 Address macros . . . 29
22.4 Example sequences . . . 29

23 Trace and Debug **30**
23.1 Commit trace . . . 30
23.2 Debug observation ports . . . 30

24 Limitations **31**

25 Integration Checklist **32**
25.1 Acceptance alignment . . . 32

26 Register Quick Reference **33**

27 Glossary **34**

28 Normative References **35**

29 Revision History **37**

List of Tables

1 Data classes i

3.1 microBee-REF32 summary 3

4.1 Functional blocks present in current RTL 4

5.1 Integer registers 5

6.1 Stage responsibilities 6

7.1 ISA enable rules 7
7.2 Execution latency (microBee-REF32) 7

8.1 Primary SoC ports 8

10.1 System address map 11

12.1 Machine-mode CSR set 14
12.2 Selected trap side effects 15
12.3 Machine interrupt causes 15

13.1 CLINT registers 16

14.1 REF32 PLIC source allocation 17
14.2 PLIC registers 17

15.1 GPIO bank registers 19

16.1 Native timer registers 20
16.2 Timer CTRL 21
16.3 Timer STATUS 21

17.1 Native UART registers 22
17.2 Native UART STATUS 23
17.3 Native UART CTRL 23
17.4 Native UART IRQ_STATUS (RW1C) 23

20.1 Primary parameters 26

21.1 Timing symbols 28

26.1 Peripheral bases 33
26.2 PLIC default source assignment (REF32 SoC fabric) 33

29.1 Document revisions 37

Chapter 1

Document Control

1.1 Identification

Document number:	microBee-TR-001
Title:	microBee Technical Reference
Revision:	P1.2
Language:	English
Product:	microBee RV32IMC soft-core microcontroller IP
Reference profile:	microBee-REF32
RTL baseline:	git `febaea8` + posted-WB retirement gate in `tmcu_core`

1.2 Normative language

The words **shall**, **shall not**, **must**, and **is required to** indicate mandatory requirements. The words **should** and **recommended** indicate preferred but non-mandatory practice. The phrase **is not supported** means the behavior is outside the product contract for the current RTL.

1.3 Related documents

- RISC-V Instruction Set Manual, Volume I (Unprivileged) — ISA encoding and base instruction semantics [1].
- RISC-V Instruction Set Manual, Volume II (Privileged) — machine-mode CSR and trap model used by this IP [2].
- Repository architecture contract `docs/architecture.md` — top-level design decisions mirrored by RTL.
- Platform header `sw/include/tmcu_platform.h` — address macros matching microBee-REF32.

RTL module and file names retain the historical `tmcu_*` prefix. The product name used in this document is microBee.

Chapter 2

Product Overview

2.1 Description

microBee is a synthesizable 32-bit RISC-V soft-core microcontroller IP intended for FPGA-based control, educational platforms that require a fixed hardware contract, and small real-time bare-metal applications. A single IP family provides RV32I, independently enableable multiply and divide units, RV32C predecode, a machine-mode trap system, local interrupt control, memory-mapped peripherals, performance monitoring, and an optional AXI4-Lite master bridge.

The core is single-issue and in-order. The application organization is a five-stage IF-ID-EX-MEM-WB pipeline. Instruction and data accesses use separate Harvard paths that share the same native valid/ready request-response protocol.

2.2 Key properties

- 32-bit little-endian RISC-V programmer's model
- RV32I base; M and C independently configurable
- Five-stage single-issue in-order pipeline
- 32×32-bit GPR file, hardwired `x0`, 2R/1W
- Optional second GPR bank on trap (`SHADOW_BANK_ENABLE`)
- Native 32-bit request-response bus with 4-bit write strobes
- Memory-mapped CLINT and compact 16-source PLIC
- SoC input `ext_irq_i[15:0]`; bit `n` maps to PLIC source ID `n+1`
- Native GPIO, general-purpose timer, UART, and performance monitor
- Optional native-to-AXI4-Lite master (`AXI_ENABLE`)
- Single hart in microBee-REF32

2.3 Delivered product items

A conforming distribution includes:

- SystemVerilog RTL and deterministic file lists
- Flat-port top/SoC modules and native interface definition
- Parameter manifest and memory map for the selected profile
- Linker script, startup code, trap template, and peripheral headers
- ELF-to-hex (or equivalent) image flow for IMEM initialization
- Verification package (unit, ISA, random, bus, and system tests)

Chapter 3

Reference Profile microBee-REF32

microBee-REF32 is the ready-to-use profile defined by this document. Profile values instantiate configurable architecture choices. A different published manifest may change memory sizes or peripheral windows; such a change requires a new profile identity when binary compatibility is affected.

Table 3.1: microBee-REF32 summary

Item	REF32 value	Notes
Hart count	1	Dual-hart is not implemented in current RTL.
ISA	RV32IMC + Zicsr + Zifencei	M and C enabled.
IMEM	64 KiB @ `0x8000_0000`	Separate BRAM/ROM port.
DMEM	64 KiB @ `0x2000_0000`	Separate BRAM port.
CLINT	`0x4000_0000`, 4 KiB	MSIP, MTIME, MTIMECMP.
PLIC	`0x4000_1000`, 4 KiB, 16 sources	3-bit priority; four trigger modes.
GPIO	32-bit @ `0x4000_2000`	`GPIO_ENABLE`.
Timer	64-bit @ `0x4000_3000`	`TIMER_ENABLE`; PLIC source 2.
Native UART	@ `0x4000_4000`	`UART_ENABLE`; PLIC source 3.
Monitor	@ `0x4000_5000`	`MONITOR_ENABLE`.
AXI window	@ `0x6000_0000`	`AXI_ENABLE`.
Board UART example	AXI UART Lite @ `0x6000_0000`	External catalog IP; not part of the microBee IP.
Nominal clock	50 MHz (FPGA REF32)	Confirmed only by timing report.
Reset vector	`0x8000_0000`	`RESET_VECTOR`.

3.1 Profile code

A synthesis output may be labeled:

`microBee-C<hart>-<ISA>-<RF>-<BUS>`

For REF32 the code is `microBee-C1-IMC-L-A` (one hart, IMC, LUT/FF register file, native+AXI). The code does not encode every parameter; the bitstream shall be archived with the parameter manifest, memory map, device part, and tool versions.

Chapter 4

Functional Description

4.1 System organization

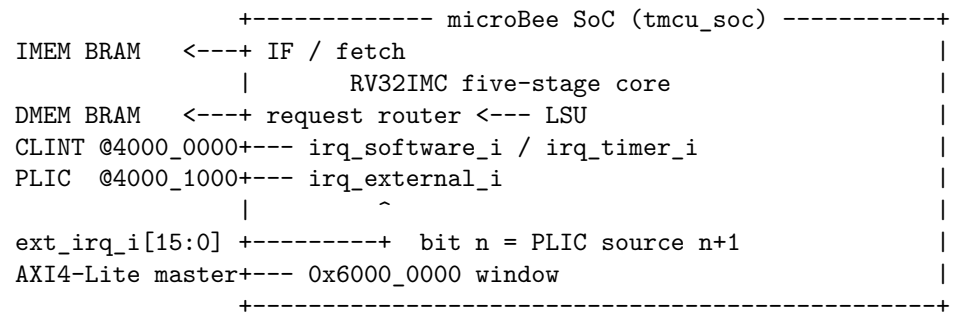


Table 4.1: Functional blocks present in current RTL

Block	Responsibility
`tmcu_core`	Pipeline, CSR/trap, native I/D ports.
`tmcu_request_router`	Address decode to memories and peripherals.
`tmcu_imem/tmcu_dmem`	Local instruction and data BRAM.
`tmcu_clint`	Software and timer interrupts; time base.
`tmcu_plic`	External interrupt controller.
`tmcu_gpio`	Memory-mapped GPIO.
`tmcu_timer`	General-purpose timer (distinct from CLINT).
`tmcu_uart`	Native UART.
`tmcu_monitor`	MMIO performance counters.
`tmcu_native_to_axi4lite`	Optional AXI4-Lite master bridge.

4.2 Clock and reset

The SoC uses a single primary clock `clk_i`. Top-level `rst_ni` is **asynchronously asserted** and **synchronously deasserted** into `clk_i` through a two-flop synchronizer (`rst_n_sync`). Internal modules use the synchronized reset.

After synchronized reset release, the first instruction fetch uses `RESET_VECTOR`. Young pipeline stages remain invalid until the first instruction response arrives. Minimum external assertion width is not architecturally timed beyond requiring a stable deassert path with a live clock; FPGA flows shall meet device reset guidelines.

4.3 Boot model

microBee does not mandate a mask ROM protocol. microBee-REF32 uses IMEM initialization from a hex/memory initialization file at synthesis or load time. Self-modifying code is not supported in the standard Harvard profile. Code and writable data shall be placed in separate physical memories by the linker script.

Chapter 5

Programmer's Model

5.1 Integer registers

The programmer-visible state comprises thirty-two 32-bit general-purpose registers, a 32-bit program counter, and the machine-mode CSR set. microBee implements machine privilege mode only. `x0` always reads as zero; writes to `x0` are ignored.

Table 5.1: Integer registers

Reg	ABI	Notes
x0	zero	Hardwired zero.
x1	ra	Return address.
x2	sp	Stack pointer.
x3–x31	gp..t6	Per RISC-V ABI; hardware treats all as GPRs.

5.2 Shadow GPR bank

When `SHADOW_BANK_ENABLE`=1, trap entry selects a second 32×32-bit GPR bank (`x1`–`x31`; `x0` remains hardwired zero in both banks). `mret` restores the normal bank. CSR trap semantics are unchanged. When the parameter is 0, a single bank is used.

This option is **not** a transparent performance switch. Firmware shall observe the following contract when the shadow bank is enabled (REF32 default in RTL is 1; a bitstream may still build with 0):

- Neither bank is reset-initialized. After reset, only the normal bank is selected; shadow contents are undefined until written.
- On the first trap, the handler executes on the shadow bank. It shall establish its own `sp` (and any other needed ABI state) before using the stack, typically from a dedicated interrupt stack symbol or by writing known constants into the shadow registers.
- Nested traps while already on the shadow bank keep the same bank selected (there is no third bank and no nest depth).
- `mret` **always** selects the normal bank. There is no hardware return to an outer handler still living on the shadow bank. Recoverable nested traps (enter outer handler → nested trap → `mret` back to outer handler on the shadow bank) are **not supported**. With the shadow bank enabled, handlers shall not rely on nested traps; treat a nested fault as fatal or disable the feature (`SHADOW_BANK_ENABLE`=0) if nesting is required.
- Application register values remain in the normal bank across the trap; the handler does not automatically see the interrupted context's GPRs.

5.3 Endianness

The data memory view is little-endian. Storing `0x1122_3344` at `0x2000_0000` places bytes `0x44`, `0x33`, `0x22`, `0x11` in ascending address order. `LB`/`LH` sign-extend; `LBU`/`LHU` zero-extend.

Chapter 6

Pipeline

6.1 Organization

The core pipeline is five stages: IF, ID, EX, MEM, and WB. The core is single-issue and in-order. Hazard detection provides forwarding, load-use interlock, stall, bubble, flush, and redirect control.

Table 6.1: Stage responsibilities

Stage	Function
IF	PC, instruction request, fetch alignment, branch redirect.
ID	RV32C decompress, decode, register read, immediate.
EX	ALU, branch resolve, mul/div start, address generation.
MEM	Load/store native request/response and memory wait.
WB	GPR/CSR writeback; instruction retirement.

6.2 Branch prediction

With `BP_ENABLE`=1 the core uses a 2-bit pattern-history table indexed by a PC fold (default). Optional `BP_GSHARE`=1 XORs the fold with a global history register of width `GHR_WIDTH`. With `BP_ENABLE`=0 every branch is predicted not-taken; program results are unchanged. Branch resolution completes in EX; misprediction flushes younger pipeline state.

6.3 Flush scope

Trap entry, `mret`, taken jump/branch redirect, and mispredict recovery clear younger pipeline state. A flush does not replay a store already accepted by the bus.

Chapter 7

Instruction Set

7.1 Supported extensions

microBee-REF32 implements RV32I, RV32M, and RV32C, plus Zicsr and Zifencei. Instruction encodings follow the ratified unprivileged specification; this chapter states only product-specific enable rules and latency.

Table 7.1: ISA enable rules

Condition	Behavior
`MUL_ENABLE`=0	`MUL*` opcodes raise illegal instruction.
`DIV_ENABLE`=0	`DIV*`/`REM*` opcodes raise illegal instruction.
Units independent	`DIV_ENABLE`=1 with `MUL_ENABLE`=0 is legal.
Disabled opcode	Shall not silently become a no-op.

7.2 Latency and throughput

Table 7.2: Execution latency (microBee-REF32)

Class	Latency	Notes
Most ALU / branch resolve	1 cycle in EX	Plus any stall/flush.
Load (local DMEM ready)	Pipeline MEM + WB	Wait extends if `rsp` delayed.
Posted local DMEM store	Accept, then WB hold	Advances to WB after accept; retirement waits for response.
Multiply (`MUL*`)	2 cycles	When `MUL_ENABLE`=1.
Divide/remainder	34 cycles	When `DIV_ENABLE`=1.
Compressed fetch	Decompressed in ID	16- and 32-bit mix in IMEM.

7.3 Fence

`FENCE` and `FENCE.I` are accepted as required by Zifencei / base fence rules implemented in the core. There is no hardware cache; fence effects are limited to ordering relative to the implemented memory system.

Chapter 8

Interfaces

8.1 Top-level / SoC ports

The shipped SoC wrapper (`tmcu_soc` / `tmcu_top`) exposes a flat-port interface suitable for Block Design packaging. Primary ports:

Table 8.1: Primary SoC ports

Port	Dir	Active	Function
`clk_i`	in	rising	System clock.
`rst_ni`	in	low	Asynchronous reset (synchronized in design).
`ext_irq_i[15:0]`	in	mode-dep.	External interrupt vector into PLIC.
GPIO bank ports	inout/in/out	—	Present when `GPIO_ENABLE`=1.
Native UART pins	in/out	—	Present when `UART_ENABLE`=1.
`M_AXI_*`	master	AXI4-Lite	Present when `AXI_ENABLE`=1.
`dbg_*`	out	—	Observation ports when `DEBUG_ENABLE`=1.

Exact port lists track the SystemVerilog top module. Integrators shall not assume additional undocumented sideband ports.

8.2 Interrupt wiring

- CLINT `irq_software_o`/`irq_timer_o` connect to core `irq_software_i`/`irq_timer_i`.
- PLIC `irq_external_o` connects to core `irq_external_i`.
- `ext_irq_i[n]` feeds PLIC source ID `n+1`.

Chapter 9

Native Request-Response Bus

9.1 Channel model

Instruction and data ports use independent request and response channels:

- Request: `req_valid`, `req_ready`, `req_write`, `req_addr`, `req_wdata`, `req_wstrb`
- Response: `rsp_valid`, `rsp_ready`, `rsp_rdata`, `rsp_err`

A transfer occurs when `valid && ready` in the same cycle. The instruction path issues reads only. The data path carries 32-bit data and a 4-bit byte strobe.

9.2 Posted store versus synchronous response

Within the local DMEM window (`DMEM_BASE`/`DMEM_MASK`), a store that is accepted on the native request channel is treated as a **posted local DMEM store**:

- At most **one** data-side transaction may be outstanding (`dmem_pending`). A new load/store request is not issued until the previous response completes.
- After request accept, the MEM stage does not stall waiting for the response of that local store; the store advances to WB.
- While the posted response is pending, WB **holds** the store (`posted_wb_hold`). Younger instructions shall not architecturally retire past that store until the response returns.
- The local DMEM slave always returns a response for every accepted request (including stores). In-range BRAM stores normally complete without error; out-of-range or read-only violations return `rsp_err`=1.
- On posted `rsp_err`, the core raises a precise store access fault: `mepc` is the PC of the posted store (`posted_pc`), and `mtval` is the store address (`posted_addr`). Younger pipeline state is flushed.

Loads and MMIO/AXI stores are not posted: the MEM stage stalls until a response that belongs to that operation arrives. A response error on those operations likewise becomes a precise load/store access fault with `mepc`/`mtval` taken from the faulting memory operation.

9.3 Outstanding requests and ordering

- Data path: maximum one outstanding request/response pair.
- Instruction path: maximum one outstanding fetch request (`fetch_pending`).
- Responses correspond to the single outstanding request on that path; reordering does not apply.

9.4 Handshake stability (implemented slave/core rules)

- A transfer occurs when `valid && ready` in the same cycle.
- While `req_valid`=1 and `req_ready`=0, the core holds the request payload stable.
- Core `dmem_rsp_ready` is asserted whenever a data response is expected (`dmem_pending`).
- Every accepted request shall receive exactly one response on the same path. Slaves implemented in this IP (local DMEM BRAM and MMIO blocks) present `rsp_valid` in the cycle after accept and hold

`rsp_rdata`/`rsp_err` until `rsp_ready` completes the handshake. Same-cycle response (accept and complete in one clock) is not required of slaves and is not used by these blocks.

- Integrator-attached native slaves shall likewise return one response per accepted request and keep the response stable while `rsp_valid`=1 and `rsp_ready`=0.

There is no architected maximum response latency beyond the connected slave behavior. Local BRAM and MMIO blocks in this IP respond in one cycle after accept under normal conditions.

Chapter 10

Memory Map

10.1 Address model

microBee presents a 32-bit byte-addressable 4 GiB logical space. Instruction and data paths are physically separate; the same numeric address may map to different physical targets on the two paths.

For **4 KiB peripheral windows** (CLINT, PLIC, GPIO, TIMER, UART, MONITOR), after the router selects the target, address bits [11:0] are the local register offset.

For **IMEM and DMEM**, indexing uses the full implemented depth. REF32 provides 64 KiB each, so bits [15:0] select the byte within the local memory image (word index uses [15:2] for 32-bit BRAM words). The low-12-bit peripheral-offset rule does not apply to IMEM/DMEM.

10.2 microBee-REF32 map

Table 10.1: System address map

Start	End	Region	Use
0x2000_0000	0x2000_FFFF	DMEM	64 KiB data BRAM.
0x2001_0000	0x3FFF_FFFF	Reserved	Error response.
0x4000_0000	0x4000_0FFF	CLINT	MSIP / MTIME / MTIMECMP.
0x4000_1000	0x4000_1FFF	PLIC	16 external sources.
0x4000_2000	0x4000_2FFF	GPIO	Native GPIO.
0x4000_3000	0x4000_3FFF	TIMER	Native timer.
0x4000_4000	0x4000_4FFF	UART	Native UART.
0x4000_5000	0x4000_5FFF	MONITOR	Performance counters.
0x4000_6000	0x4000_6FFF	Reserved	Error in REF32 (no mailbox RTL).
0x4000_7000	0x5FFF_FFFF	Reserved	Error response.
0x6000_0000	0x6FFF_FFFF	AXI window	External AXI4-Lite slaves.
0x7000_0000	0x7FFF_FFFF	Reserved	Error response.
0x8000_0000	0x8000_FFFF	IMEM	64 KiB instruction BRAM/ROM.
0x8001_0000	0xFFFF_FFFF	Reserved	Profile-dependent / error.

10.3 Decode priority

`tmcu_request_router` checks regions in this order: CLINT → PLIC → GPIO → TIMER → UART → MONITOR → DMEM → AXI → error. When `MONITOR_ENABLE`=0, a MONITOR address hit still returns error (it does not fall through to DMEM). Disabled peripheral windows similarly decode as error rather than aliasing into another target.

10.4 Data-path access to the IMEM window

The data-side router decode targets are CLINT, PLIC, GPIO, TIMER, UART, MONITOR, DMEM, and AXI. The IMEM window (`0x8000_0000`) is **not** a data-path slave in the shipped `tmcu_soc` configuration:

- `IMEM_ALIAS_ENABLE` defaults to 0 and is not enabled in `tmcu_soc`. Loads/stores to the IMEM window therefore decode as error (or fall through only if an integrator remaps the window).
- Instruction fetch remains the only normative path into the IMEM BRAM.
- Simulation TBs may enable a DV-only IMEM data alias (`+dmem_imem_alias` / router `IMEM_ALIAS_ENABLE`=1). That path is not part of the REF32 SoC product contract.

10.5 Linker placement

The REF32 product linker (`sw/linker/tmcu_fpga_arty.ld`) and the Embench board linker place `.text` in IMEM and `.rodata`/`.srodata`/`.data`/`.bss`/heap/stack in DMEM. Stack grows down from the top of DMEM and remains 16-byte aligned per the ILP32 ABI.

Some DV linkers (riscv-dv / arch-test) may still co-locate `.rodata` with `.text` in IMEM; those flows require a TB IMEM data alias and are not the shipped SoC contract.

10.6 Startup image and `.data` initialization

Local BRAMs are loaded from synthesis/simulation init images (`IMEM_INIT_FILE`, `DMEM_INIT_FILE`) or by a board programmer before release from reset. Startup that copies `.data` at run time shall read the load image from a data-reachable region (DMEM or an integrator-provided alias), not from the default IMEM window. When the DMEM init file already contains the relocated `.data` image, a run-time copy from IMEM is unnecessary.

Chapter 11

Local Memory

11.1 IMEM

IMEM is a local BRAM/ROM target that accepts **instruction-fetch** read traffic only under the shipped SoC contract. Capacity is set by `IMEM_WORDS` (REF32: 16384 words = 64 KiB) at base `0x8000_0000`. Addresses outside the implemented depth that still decode to the IMEM fetch window produce an access fault; they are not aliased to another memory.

Data-side loads and stores to the IMEM numeric window are not routed to the IMEM BRAM in default `tmcu_soc` (see Memory Map). Do not assume `.rodata` in IMEM is readable via `LW/LB`.

11.2 DMEM

DMEM supports byte, halfword, and word reads and writes. Capacity is set by `DMEM_WORDS` (REF32: 16384 words = 64 KiB) at base `0x2000_0000`. Unimplemented locations in the window produce access faults. Readable constants and any run-time `.data` source image used by startup shall be placed here unless an integrator documents a non-default IMEM data path.

11.3 Coherence

The instruction fetch path does not provide automatic coherence with data writes. Because self-modifying code is not supported in the standard Harvard profile, this restriction does not affect conforming software. A bootloader that writes IMEM shall complete those writes through a SoC-level write path before releasing the core.

Chapter 12

CSR and Traps

12.1 Implemented machine CSRs

Table 12.1: Machine-mode CSR set

Addr	CSR	Access	Function
0x300	`mstatus`	MRW/WARL	Global interrupt enable state.
0x304	`mie`	MRW/WARL	Software, timer, external enables.
0x305	`mtvec`	MRW/WARL	Trap vector base; Direct mode only.
0x340	`mscratch`	MRW	Scratch for trap software.
0x341	`mepc`	MRW/WARL	Exception PC.
0x342	`mcause`	MRW	Interrupt bit and cause code.
0x343	`mtval`	MRW	Faulting address or related value.
0x344	`mip`	MRW/WARL	Pending bits (wired where applicable).
0xB00	`mcycle`	MRW	Cycle counter low.
0xB80	`mcycleh`	MRW	Cycle counter high.
0xB02	`minstret`	MRW	Retired-instruction counter low.
0xB82	`minstreth`	MRW	Retired-instruction counter high.

CSRs outside this table are not implemented as product features. Access to an unimplemented CSR raises illegal instruction as implemented by the core.

12.2 WARL and field behavior (implemented)

- `mstatus`: writable bits are MIE[3] and MPIE[7]. MPP[12:11] is hardwired to `2'b11` (M-mode). Reset value `0x0000_1800`.
- `mie`: only MSIE[3], MTIE[7], MEIE[11] are writable; other bits read as zero.
- `mip`: MSIP/MTIP/MEIP read as software-writable shadow OR the corresponding hardware lines (`irq_software_i`, `irq_timer_i`, `irq_external_i`). Software may write the shadow bits; clearing a hardware-driven pending requires clearing the source (CLINT/PLIC), not only writing `mip`.
- `mepc`: bit [0] is forced to 0 on write and trap entry (IALIGN=16 with C). Bit [1] is writable (compressed-aligned PC).
- `mtvec`: Direct mode only; Vectored mode is not supported.

12.3 Exception `mepc/mtval` summary

Table 12.2: Selected trap side effects

Condition	`mepc`	`mtval`
Illegal instruction	Faulting PC	Instruction bits.
Instruction access fault	Faulting PC	Faulting PC.
Load/store misaligned or access fault	Faulting PC	Faulting address.
Posted local DMEM store `rsp_err`	Posted store PC	Posted store address.
ECALL / EBREAK	Faulting PC	0 (as implemented).
Interrupt	PC of interrupted fetch point	0.

12.4 Interrupt causes

Table 12.3: Machine interrupt causes

Cause	Code	Source
Machine software	`0x8000_0003`	CLINT MSIP.
Machine timer	`0x8000_0007`	CLINT `mtime >= mtimecmp`.
Machine external	`0x8000_000B`	PLIC `irq_external`.

Acceptance requires the corresponding `mie` bit and `mstatus.MIE`. Synchronous exceptions use the standard RISC-V cause codes implemented by the decoder and LSU (illegal instruction, misaligned / access fault, ecall, ebreak, and related cases present in RTL).

12.5 Trap entry and return

On trap, the core updates `mepc`, `mcause`, `mtval` as applicable, updates `mstatus`, and redirects fetch to `mtvec`. `mret` restores prior state and resumes at `mepc`. Younger pipeline instructions are flushed.

12.6 Counter reads

64-bit `mcycle` and `minstret` shall be read with a high-low-high sequence (or equivalent consistent method) to avoid wrap tearing on RV32.

Chapter 13

CLINT

13.1 Overview

The REF32 CLINT (`tmcu_clint`) provides a software-interrupt bit, a 64-bit timer compare value, and a 64-bit free-running time base for a single hart. Base address `0x4000_0000`; 4 KiB window (`MASK=0xFFFF_F000`). Local offset is address bits [11:0].

Table 13.1: CLINT registers

Offset	Register	Access	Description
0x000	`MSIP`	RW	Software interrupt pending; bit 0 only.
0x400	`MTIMECMP_LO`	RW	Compare low.
0x404	`MTIMECMP_HI`	RW	Compare high.
0xFF8	`MTIME_LO`	RW	Time base low.
0xFFC	`MTIME_HI`	RW	Time base high.

Misaligned accesses return `rsp_err`. Undefined word-aligned offsets read as 0 and ignore writes without bus error.

13.2 MSIP and timer interrupt

`MSIP` bit 0 is writable; bits [31:1] are RAZ/WI. `irq_software_o` equals that bit. Cause code `0x8000_0003`.

`irq_timer_o` is asserted while `mtime >= mtimecmp` (level compare). Cause code `0x8000_0007`.

13.3 Reset and time base

After reset, `mtime`=0, `msip`=0, and `mtimecmp`=`0xFFFF_FFFF_FFFF_FFFF`, so no timer interrupt is pending at reset. `mtime` increments by one each core clock cycle. A write to `MTIME_*` in the same cycle replaces the auto-increment value for that cycle.

13.4 64-bit compare update order

To avoid a spurious early interrupt on RV32:

1. Write `0xFFFF_FFFF` to `MTIMECMP_LO`.
2. Write the new high word to `MTIMECMP_HI`.
3. Write the new low word to `MTIMECMP_LO`.

Chapter 14

PLIC

14.1 Overview

The REF32 PLIC (`tmcu_plic`) uses a compact 4 KiB map at `0x4000_1000`. It preserves the RISC-V software model (priority, pending, enable, threshold, claim/complete) without a SiFive-scale address layout. REF32 uses 16 sources and 3-bit priority. Source ID 0 means "no interrupt" and is not a physical source.

Claim and complete use **separate** addresses (unlike some ratified PLIC layouts that share one address). Software shall treat this as a microBee platform rule.

14.2 REF32 source allocation

SoC input `ext_irq_i[n]` feeds PLIC source ID `n+1`. Native timer and native UART IRQs are **OR-combined** onto the same PLIC inputs as the corresponding `ext_irq_i` bits (defaults `TIMER_IRQ_ID`=2, `UART_IRQ_ID`=3):

Table 14.1: REF32 PLIC source allocation

Source ID	Combined input	Notes
0	—	Reserved ("no interrupt").
1	`ext_irq_i[0]`	Board example: AXI UART Lite.
2	`ext_irq_i[1]` OR native timer IRQ	Shared line when TIMER_ENABLE.
3	`ext_irq_i[2]` OR native UART IRQ	Shared line when UART_ENABLE.
4–16	`ext_irq_i[3:15]`	Integrator; tie unused inputs.

If both the pad and the native peripheral assert source 2 or 3, software sees a single PLIC source; it cannot distinguish which contributor fired without additional status reads in the peripheral.

Because sources 2 and 3 are OR-combined before the gateway, a rising-edge mode sees only transitions of the OR result. If one contributor is already high, a rising edge on the other contributor does **not** create a new rising edge on the shared line. For native timer/UART, prefer level-sensitive mode (typically active-high, code 11) unless the pad protocol requires edges.

Unused `ext_irq_i` contributors on a shared OR line (sources 2 and 3 when the native peripheral is used) shall be tied to **0**. The pad is OR-combined with the native IRQ, so tying the unused pad to 1 holds the combined line high and masks native IRQ transitions regardless of trigger mode.

Table 14.2: PLIC registers

Offset	Register	Access	Description
0x000	`CAPABILITY`	RO	[7:0] sources; [11:8] priority bits.
0x004+4(i-1)	`PRIORITY[i]`	RW	Source i priority, $i = 1..N$.
0x100	`PENDING`	RO	Bit k = source k pending; bit 0 reserved.

Offset	Register	Access	Description
0x104	SOURCE_MODE	RW	Two bits per source trigger mode.
0x108	INPUT_STATE	RO	Post-synchronizer levels.
0x10C	RESET_LEVEL	RO	Reset seed for edge history.
0x180	ENABLE	RW	Per-source enable; bit 0 reserved.
0x300	THRESHOLD	RW	Hart-0 priority threshold.
0x304	CLAIM	RO	Highest eligible ID (side effect).
0x308	COMPLETE	WO	Completed source ID.

After reset:

- CAPABILITY reads 0x0000_0310 (16 sources, 3 priority bits).
- All PRIORITY[i]=0; ENABLE=0; THRESHOLD=0.
- SOURCE_MODE=IRQ_MODE_RESET (default all zeros = rising edge for every source).
- PENDING and in-service bits are clear.

14.3 SOURCE_MODE and INPUT_STATE packing

For source ID i ($i = 1..N$):

- SOURCE_MODE bits $[(i-1) \times 2+ : 2]$ hold the 2-bit mode.
- INPUT_STATE bit $(i-1)$ is the post-synchronizer level for that source (bit 0 = source 1).
- ENABLE/PENDING bit k corresponds to source k; bit 0 is reserved and reads 0.

14.4 Selection rules

- Eligible: pending=1, enable=1, priority $\neq 0$, priority $>$ threshold.
- Highest priority wins; ties broken by lowest ID.
- Claim returns the ID, clears pending, sets in-service.
- Complete frees in-service for the written ID.
- Priority 0 never interrupts.

14.5 Trigger modes

Each source has a 2-bit SOURCE_MODE field:

Code	Behavior
00	Rising edge.
01	Falling edge.
10	Active-low level.
11	Active-high level.

Mode changes should occur only while the source is disabled and not in-service. Gateway holds a single event while pending or in-service; level sources may become pending again after complete if still active.

14.6 Input synchronization

With IRQ_SYNC_ENABLE=1, each channel uses a 2-FF synchronizer (ASYNC_REG). An asynchronous pulse shall be at least two core clocks wide to be observed reliably.

Chapter 15

GPIO

15.1 Overview

Native GPIO provides `GPIO_WIDTH`-bit banks (REF32: one 32-bit bank, base `0x4000_2000`). FPGA I/O standard, drive, slew, and pulls are board constraints, not IP electrical guarantees.

15.2 Register map

Bank stride is 0x20. Bank b adds offset $0x20 \times b$.

Table 15.1: GPIO bank registers

Offset	Register	Access	Description
0x00	`IN`	RO	Synchronized input level.
0x04	`OUT`	RW	Output data.
0x08	`OUT_SET`	WO	Atomic set of written-1 bits.
0x0C	`OUT_CLR`	WO	Atomic clear of written-1 bits.
0x10	`OUT_TGL`	WO	Atomic toggle of written-1 bits.
0x14	`OE`	RW	1 = output enable; 0 = input/Hi-Z.
0x18	`VALID`	RO	Implemented channel mask.
0x1C	Reserved	—	Reads 0; writes ignored.

After reset, `OUT`=0 and `OE`=0. Bits above the physical width read 0 and ignore writes (`VALID`=0).

15.3 Input synchronization

Asynchronous pin inputs are synchronized with at least two flip-flops in the SoC wrapper. This reduces metastability propagation but does not guarantee capture of narrow pulses. Typical added latency is two `clk_i` cycles.

Chapter 16

General-Purpose Timer

16.1 Relation to CLINT

The native timer at `0x4000_3000` is independent of the CLINT `mtime`/`mtimecmp` time base. Machine timer interrupts used for RISC-V `MTIP` come from CLINT. The native timer may drive a PLIC source (REF32: source 2, OR-combined with `ext_irq_i[1]`) when enabled in the SoC interrupt fabric.

16.2 Operation

- The counter counts **up**.
- A tick occurs when ENABLE and `prescale_cnt >= PRESCALE` (tick every PRESCALE+1 clocks when PRESCALE=0 means every clock).
- On each tick, COUNT becomes COUNT+1, then compare uses **COUNT >= CMPx** (after the increment).
- CMP0 match with AUTO_RELOAD loads PERIOD into COUNT in the same tick decision.
- IRQ output: (STATUS.CMP0 ∧ CTRL.IRQ0_EN) ∨ (STATUS.CMP1 ∧ CTRL.IRQ1_EN).
- STATUS.CMPx bits are sticky; write-1-to-clear.
- If a compare sets CMPx and software clears the same bit in the same cycle, the W1C clear wins (status ends cleared for that cycle).
- Reset values: CTRL=0, PRESCALE=0, COUNT=0, PERIOD=0, STATUS CMPx=0; CMP0=CMP1=`0xFFFF_FFFF_FFFF_FFFF`.
- 64-bit COUNT/CMP/PERIOD updates are not atomic across LO/HI; software shall use disable or a consistent update order when needed.

16.3 Registers

Table 16.1: Native timer registers

Offset	Register	Access	Description
0x00	`CTRL`	RW	Enable, auto-reload, IRQ enables.
0x04	`PRESCALE`	RW	Tick every PRESCALE+1 cycles.
0x08	`COUNT_LO`	RW	Counter low.
0x0C	`COUNT_HI`	RW	Counter high.
0x10	`CMP0_LO`	RW	Compare 0 low.
0x14	`CMP0_HI`	RW	Compare 0 high.
0x18	`CMP1_LO`	RW	Compare 1 low.
0x1C	`CMP1_HI`	RW	Compare 1 high.
0x20	`PERIOD_LO`	RW	Auto-reload period low.
0x24	`PERIOD_HI`	RW	Auto-reload period high.
0x28	`STATUS`	RW1C	Sticky compare pending; running.
0x2C	`CAPABILITY`	RO	Width/channel capability.

16.4 CTRL and STATUS bits

Table 16.2: Timer CTRL

Bit	Field	Description
0	ENABLE	Run counter.
1	AUTO_RELOAD	On CMP0 match, load PERIOD.
8	IRQ0_EN	Compare 0 interrupt enable.
9	IRQ1_EN	Compare 1 interrupt enable.

Table 16.3: Timer STATUS

Bit	Field	Description
0	CMP0	Sticky compare 0; write 1 to clear.
1	CMP1	Sticky compare 1; write 1 to clear.
8	RUNNING	Read-only mirror of ENABLE.

Chapter 17

UART

17.1 Native UART versus board example

Native UART is implemented in RTL at `0x4000_4000` when `UART_ENABLE`=1 (PLIC source 3 in REF32). Separately, microBee-REF32 FPGA board examples may attach an external AXI UART Lite (Xilinx PG142) in the AXI window at `0x6000_0000` (PLIC source 1). AXI UART Lite is **not** part of the microBee IP and shall be documented from the vendor IP guide when used.

17.2 Native UART function

Full-duplex 8N1. REF32 uses 16-entry TX and RX FIFOs (`UART_FIFO_DEPTH`). The bit period is `BAUDDIV` system clocks when `BAUDDIV`≥8:

$$\text{baud} = f_{\text{clk}}/\text{BAUDDIV}$$

Reset `BAUDDIV`=434 (approximately 115200 baud at 50 MHz). `BAUDDIV`=0 disables baud ticks. Values 1..7 do not start the serializer/sampler (enable path requires `BAUDDIV`≥8).

RX uses a 4×-faster sample clock derived as $\lfloor \text{BAUDDIV}/4 \rfloor$ for start-bit detection and mid-bit sampling (implementation detail of the oversampler).

Table 17.1: Native UART registers

Offset	Register	Access	Description
0x00	`DATA`	RW	Write pushes TX; read pops RX.
0x04	`STATUS`	RO	FIFO and error flags.
0x08	`CTRL`	RW	TX/RX and IRQ enables.
0x0C	`BAUDDIV`	RW	Clocks per bit; see formula above.
0x10	`IRQ_STATUS`	RW1C	Sticky IRQ/error causes.
0x14	`FIFO_LEVEL`	RO	RX[7:0], TX[23:16] occupancy.
0x18	`CAPABILITY`	RO	FIFO depth / format.

Reading `DATA` while RX empty returns 0 and does not pop.

Writing `DATA` while TX is full does **not** stall the accepted write until space exists. The RTL behavior is:

1. The write request is accepted (`req_valid && req_ready`).
2. The byte is **not** pushed into the TX FIFO (data is discarded).
3. A normal success response is still returned for that request.
4. `write_stall` then deasserts `req_ready` for one cycle, so a immediately following request may be held briefly on the bus.

Software shall therefore poll `TX_FULL`=0 (or use TX IRQ) before every `DATA` write; otherwise bytes can be lost. On RX overrun, the new byte is dropped and the overrun sticky bit is set (FIFO contents retained).

17.3 STATUS and CTRL (native)

Table 17.2: Native UART STATUS

Bit	Field	Description
0	RX_VALID	RX FIFO not empty.
1	RX_FULL	RX FIFO full.
2	TX_EMPTY	TX FIFO empty.
3	TX_FULL	TX FIFO full.
4	TX_BUSY	Serializer active.
8	OVERRUN	Overflow.
9	FRAME_ERR	Bad stop bit.

Table 17.3: Native UART CTRL

Bit	Field	Description
0	TX_EN	Transmitter enable.
1	RX_EN	Receiver enable.
8	RX_IRQ_EN	RX data IRQ enable.
9	TX_IRQ_EN	TX empty IRQ enable.
10	ERR_IRQ_EN	Error IRQ enable.
16	LOOPBACK	TX to RX loopback.

17.4 IRQ_STATUS and FIFO_LEVEL

Table 17.4: Native UART IRQ_STATUS (RW1C)

Bit	Field	Description
0	IRQ_TX	Sticky TX IRQ cause; write 1 clears.
1	IRQ_RX	Sticky RX IRQ cause; write 1 clears.
2	IRQ_ERR	Sticky error IRQ cause; write 1 clears.
8	OVERRUN	Also clears STATUS.OVERRUN when written 1.
9	FRAME_ERR	Also clears STATUS.FRAME_ERR when written 1.

Read data for `IRQ_STATUS` returns bits [2:0] only (`IRQ_TX` at bit 0, `IRQ_RX` at bit 1, `IRQ_ERR` at bit 2); bits 8–9 are write-1-to-clear side effects on the STATUS sticky errors.

`FIFO_LEVEL` packing in RTL:

7:0 = RX occupancy (`rx_count`)
23:16 = TX occupancy (`tx_count`)

- other bits read as 0

Occupancy values are the live FIFO element counts (0..`UART_FIFO_DEPTH`). With the current pointer-width implementation, supported depths are powers of two in 2..16 (REF32 uses 16). A depth of 256 is not implemented by the present FIFO pointer logic; do not rely on saturating an 8-bit level at 256.

Chapter 18

Performance Monitor

18.1 Overview

When `MONITOR_ENABLE`=1, MMIO counters are available at `0x4000_5000`. Events are probed inside the core and counted in `tmcu_monitor`. CSR `mcycle`/`minstret` remain separate.

18.2 Control

`MON_CTRL` at offset 0x000:

- bit 0 `ENABLE`
- bit 1 `FREEZE`
- bit 2 `CLEAR` (write-1 pulse)
- bit 3 `SNAPSHOT` (write-1 pulse)

Priority: reset → CLEAR → SNAPSHOT → count while ENABLE and not FREEZE. Shadow copies live at live_offset + 0x100.

18.3 Counter offsets (live)

Selected offsets (see `tmcu_platform.h` for the full set):

Off	Name	Notes
0x004	CYCLES	Window cycles.
0x008	RETIRED	Retired instructions.
0x00C	STALL	Stall cycles (may overlap other waits).
0x010	FLUSH	Pipeline flushes.
0x020	BRANCH	Branches.
0x028	MISPREDICT	Mispredictions.
0x04C	CAPABILITY	Feature bitmap (MAP revision 2).
0x064	SPEC_REDIRECT	Speculative taken redirect (BP on).

Stall, IMEM wait, DMEM wait, and similar counters may overlap; reports shall not sum them as mutually exclusive.

Retirement used by MONITOR `RETIRED` and CSR `minstret` is a single pulse per instruction: `mem_wb_valid && !exception && !posted_wb_hold`. While a posted local DMEM store waits in WB, retirement is suppressed; the instruction increments the counters once when the response completes without error. CPI/IPC derived from these counters is meaningful across varying DMEM response latency.

Chapter 19

AXI4-Lite Master

19.1 Scope

When `AXI_ENABLE`=1, native data requests that decode into the AXI window (`0x6000_0000` in REF32) are converted by `tmcu_native_to_axi4lite` into an AXI4-Lite master. The core never speaks AXI directly.

19.2 Bridge contract (implemented)

- Address/data width: 32-bit; `WSTRB` is 4 bits.
- `AWPROT`/`ARPROT` are driven as `3'b000`.
- Native address is presented unchanged on AXI (`0x6000_1234` appears as `0x6000_1234` on `AWADDR`/`ARADDR`).
- Single outstanding native transaction; AW/W may complete in either order before B; reads wait for R.
- `BRESP`/`RRESP` other than OKAY map to native `rsp_err`=1 and therefore load/store access fault.
- AXI timeout is not implemented in this bridge. If an integrator adds a timeout, duration and fault mapping belong in the product profile manifest.

Chapter 20

Configuration Parameters

Table 20.1: Primary parameters

Parameter	Range	REF32	Function
RESET_VECTOR	32-bit	0x8000_0000	First fetch address.
MTVEC_RESET	aligned	0x8000_0000	Direct trap base after reset.
SHADOW_BANK_ENABLE	0/1	1 (default)	Second GPR bank on trap; see programmer's model.
MUL_ENABLE	0/1	1	2-cycle multiply; else illegal.
DIV_ENABLE	0/1	1	34-cycle divide; else illegal.
MONITOR_ENABLE	0/1	1	MMIO counters; else error.
AXI_ENABLE	0/1	1	AXI master window; else error.
BP_ENABLE	0/1	1	Branch predictor.
BP_ENTRIES	power of 2	128	PHT entries (RTL default).
BP_GSHARE	0/1	0	Gshare index enable.
GHR_WIDTH	see note	5	Used when BP_GSHARE=1.
IMEM_WORDS/DMEM_WORDS	>0	16384	Local BRAM words.
GPIO_ENABLE	0/1	1	Native GPIO window.
GPIO_WIDTH	1..32 per bank	32	Bank data width (32-bit MMIO).
TIMER_ENABLE	0/1	1	Native timer window.
UART_ENABLE	0/1	1	Native UART window.
CLINT_ENABLE	0/1	1	CLINT window / IRQ.
PLIC_ENABLE	0/1	1	PLIC window / IRQ.
PLIC_SOURCES	1..16	16	External sources; SOURCE_MODE packing limits layout to 16.
PLIC_PRIO_BITS	1..8	3	Priority field width.
IRQ_SYNC_ENABLE	0/1	1	Per-source 2-FF sync.
UART_FIFO_DEPTH	power of 2, 2..16	16	Native UART FIFO (current pointer width).
TRACE_ENABLE	0/1	0	Commit trace ports.
DEBUG_ENABLE	0/1	configurable	dbg_* observe ports.

When BP_GSHARE=1, GHR_WIDTH shall satisfy $1 \leq$ GHR_WIDTH $\leq \log_2$(BP_ENTRIES). With REF32 defaults (BP_ENTRIES=128, BP_GSHARE=0), the GHR_WIDTH=5 value is retained in the parameter list but is not used for indexing until Gshare is enabled.

20.1 Dependencies

1. Multiply and divide enables are independent.
2. Disabled ISA opcodes raise illegal instruction.

3. Disabled peripheral windows decode as error, not as alias.
4. With `IRQ_SYNC_ENABLE`=1, asynchronous pulses need $\geq$2 clocks.
5. Memory-map or ISA changes that break binaries require a new profile ID.

Chapter 21

Clocking, Timing, and Resources

21.1 Logical timing

Table 21.1: Timing symbols

Symbol	Definition	Source
`tCK`	Minimum clock period	Place-and-route.
`fMAX`	Highest closed frequency	1/tCK.
`tSU/tH`	Top-level setup/hold	Constraints.
`tCO`	Clock-to-output	Implementation.

The IP alone does not guarantee pin timing in nanoseconds.

21.2 REF32 operating point

REF32 is prepared for a nominal 50 MHz clock. That frequency is an approved operating point only when the target part and speed grade are stated, all relevant paths meet a 20 ns (or tighter) constraint, WNS is non-negative, and CDC/I/O checks pass for the same parameter manifest.

21.3 Resource and electrical limits

LUT, FF, BRAM, DSP, fMAX, WNS, and power shall be taken from the implementation report for each published bitstream. Core supply, I/O voltage, absolute maximum ratings, ESD, and operating temperature are **not applicable** as microBee IP guarantees; use the FPGA/device data sheet and board design.

Chapter 22

Software Integration Contract

22.1 Toolchain

Bare-metal software targets a 32-bit RISC-V ILP32 environment with `-march` matching the enabled ISA (REF32: `rv32imc`). Standard GCC or LLVM RISC-V toolchains producing ELF images are supported.

22.2 Startup order

1. Reset releases; fetch begins at `RESET_VECTOR`.
2. Place readable constants in DMEM (or enable a documented IMEM data alias). Do not rely on data loads from the default IMEM window.
3. Startup may copy `.data` into DMEM from a data-reachable load image, or rely on `DMEM_INIT_FILE`; then clear `.bss`.
4. Set `mtvec`, application stack pointer, and interrupt enables. If `SHADOW_BANK_ENABLE`=1, also define how the trap handler obtains its shadow-bank `sp`.
5. Enter application `main`.

22.3 Address macros

Firmware shall use a single generated or maintained header matching the bitstream map. The repository header `sw/include/tmcu_platform.h` defines REF32 bases, including:

```
TMCU_IMEM_BASE          0x80000000
TMCU_DMEM_BASE          0x20000000
TMCU_CLINT_BASE         0x40000000
TMCU_PLIC_BASE          0x40001000
TMCU_GPIO_BASE          0x40002000
TMCU_TIMER_BASE         0x40003000
TMCU_NATIVE_UART_BASE   0x40004000
TMCU_MONITOR_BASE       0x40005000
TMCU_AXI_BASE           0x60000000
```

22.4 Example sequences

CLINT compare updates shall follow the low=max, high, low order in the CLINT chapter. PLIC handlers shall claim, service, and complete using the separate CLAIM/COMPLETE addresses. Native UART transmit shall poll until `TX_FULL`=0 (or use TX IRQ), then write `DATA`. Waiting only for `TX_EMPTY` under-utilizes the TX FIFO.

Chapter 23

Trace and Debug

23.1 Commit trace

When `TRACE_ENABLE`=1, the core exposes a flat commit-trace interface describing retired instructions, PC, register writes, memory operations, and trap events as implemented for verification. On an interrupt event the instruction field is zero and the PC field carries `mepc`.

23.2 Debug observation ports

When `DEBUG_ENABLE`=1, SoC/core `dbg_*` ports export selected micro-architectural signals for simulation or FPGA ILA observation. These ports are not a RISC-V External Debug specification implementation and do not constitute a JTAG debug module unless separately integrated by the customer.

Chapter 24

Limitations

The following are **not** product features of the current RTL and shall not be assumed by integrators:

- Dual-hart / mailbox hardware (not implemented in this revision; window `0x4000_6000` returns error in REF32).
- User or Supervisor privilege modes; PMP; hypervisor extensions.
- A, F, or D ISA extensions.
- Caches or hardware coherence.
- Self-modifying code on the standard Harvard profile.
- Vectored `mtvec` mode.
- Recoverable nested traps while `SHADOW_BANK_ENABLE`=1 (`mret` always selects the normal bank).
- Packaged-device absolute maximum electrical ratings.
- AXI UART Lite or other catalog peripherals inside the IP package.
- Built-in AXI transaction timeout.

Posted local DMEM stores leave MEM after request accept and sit in WB until the response returns (`posted_wb_hold`). Younger instructions do not architecturally retire past that store during the hold. A late `rsp_err` is a precise store access fault. Software may observe the store as "accepted" on the bus before retirement completes; that is not the same as fire-and-forget completion relative to younger instructions.

Chapter 25

Integration Checklist

Integrators shall verify the following before accepting a bitstream or simulation configuration:

1. `clk_i` / `rst_ni` polarity and synchronization match the board reset scheme.
2. `RESET_VECTOR` and IMEM initialization image agree.
3. `ext_irq_i` width is 16; unused inputs are tied appropriately.
4. PLIC source mapping for board IRQs is documented (bit $n \rightarrow$ source $n + 1$).
5. Memory map in software headers matches the router bases/masks.
6. Disabled feature-enable parameters (`GPIO_ENABLE`, `UART_ENABLE`, and similar) leave their windows unused by software.
7. Timing closure and CDC reports are archived with the parameter manifest.

25.1 Acceptance alignment

Repository acceptance flows (`docs/integration-tests.md`) exercise CLINT MSIP/MTIP, PLIC priority/claim modes, concurrent sources, AXI window accesses, and unmapped faults. Customer acceptance should cover the same classes of behavior for the integrated system, using customer stimulus rather than requiring this document to define laboratory procedures.

Chapter 26

Register Quick Reference

Base addresses are microBee-REF32 values.

Table 26.1: Peripheral bases

Block	Base	Size	Chapter
CLINT	`0x4000_0000`	4 KiB	CLINT
PLIC	`0x4000_1000`	4 KiB	PLIC
GPIO	`0x4000_2000`	4 KiB	GPIO
Timer	`0x4000_3000`	4 KiB	Timer
Native UART	`0x4000_4000`	4 KiB	UART
Monitor	`0x4000_5000`	4 KiB	Monitor
AXI window	`0x6000_0000`	256 MiB decode	AXI
DMEM	`0x2000_0000`	64 KiB	Local memory
IMEM	`0x8000_0000`	64 KiB	Local memory

Table 26.2: PLIC default source assignment (REF32 SoC fabric)

Source	Typical connection	Notes
1	Board AXI UART Lite IRQ	External IP example.
2	`ext_irq_i[1]` OR native timer	Shared OR; unused pad tie 0.
3	`ext_irq_i[2]` OR native UART	Shared OR; unused pad tie 0.
4–16	Integrator	Tie unused pads inactive (shared OR pads to 0).

Detailed bit fields appear in the peripheral chapters. Production firmware shall prefer generated headers over duplicating tables.

Chapter 27

Glossary

CLINT Core-local interruptor: MSIP and machine timer compare.
Harvard Separate instruction and data memory ports.
microBee-REF32 Default product profile defined in this document.
Native bus Valid/ready request-response protocol used by the core.
PLIC Platform-level interrupt controller (compact 16-source map).
Posted store Local DMEM store accepted without waiting for response.
WARL Write-any, read-legal CSR field behavior.

Chapter 28

Normative References

Bibliography

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA* (ratified version applicable to the product release).

[2] RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture* (ratified version applicable to the product release).

Chapter 29

Revision History

Table 29.1: Document revisions

Revision	Date	Notes
P1.0	2026-09-29	Initial English technical reference (microBee-TR-001). Aligned to shipped RTL and architecture contract; dual-hart omitted.
P1.1	2026-09-29	Review fixes: IMEM/`.rodata` data-path contract; UART TX-full and IRQ_STATUS; BP_ENTRIES/GHR; shadow-bank firmware rules; PLIC packing/OR-edge note; FIFO depth; posted-store wording; traceability.
P1.2	2026-09-29	RTL: gate `minstret`/MONITOR retire on `!posted_wb_hold`; nested shadow `mret` unsupported; shared PLIC OR unused pad tie-0; REF32 linker moves `.rodata` to DMEM.

End of microBee-TR-001.